

Министерство образования Республики Беларусь
Учреждение образования «Белорусский государственный университет информатики и радиоэлектроники»

Факультет компьютерных систем и сетей
Кафедра Информатики
Дисциплина «Методы защиты информации»

ОТЧЁТ
к лабораторной работе №5
на тему
«ХЕШ-ФУНКЦИИ»

БГУИР 6-05-0612-02 098

Выполнил студент группы 353502
ПЕТРОЧЕНКО Михаил Максимович

(дата, подпись студента)

Проверил ассистент кафедры информатики
ГЕРЧИК Артём Вадимович

(дата, подпись преподавателя)

Минск 2026

СОДЕРЖАНИЕ

1	Цель работы	3
2	Ход работы	4
2.1	Алгоритм SHA-1	4
2.2	Алгоритм ГОСТ 34.11-2012	5
2.3	Хеширование текста	6
2.4	Лавинный эффект	7
2.5	Работа с файлами	8
	Заключение	9
	Приложение А (обязательное) Листинг программного кода	10
	Приложение Б (обязательное) Блок-схемы алгоритмов	23

1 ЦЕЛЬ РАБОТЫ

Целью данной лабораторной работы является изучение принципов построения криптографических хеш-функций на примере алгоритма SHA-1 и алгоритма ГОСТ 34.11-2012 «Стрибог». Задачи включают изучение структуры и математических основ обоих алгоритмов, проектирование и реализацию программного средства, вычисляющего хеш-код SHA-1 длиной 160 бит и хеш-коды ГОСТ 34.11-2012 длиной 256 и 512 бит, а также тестирование разработанного программного средства.

Конечным результатом данной лабораторной работы должна быть разработанная программа, вычисляющая хеш-функции SHA-1 и ГОСТ 34.11-2012 как для текста, вводимого с клавиатуры, так и для файлов произвольного содержания.

2 ХОД РАБОТЫ

В ходе выполнения работы было создано программное средство на языке программирования *Rust*. Хеш-функции реализованы в отдельных модулях `hashing::sha1` и `hashing::gost34_11`, сторонние библиотеки не используются. Взаимодействие с пользователем осуществляется через текстовое меню, общее с предыдущими лабораторными работами: оно позволяет выбрать источник данных (текст или файл), вычислить хеш-код одним из алгоритмов или всеми сразу и запустить проверку на контрольных примерах. Общий вид меню программы и результат проверки представлены на рисунке 2.1.

```
Hash functions - input/output: text
1) SHA-1          2) GOST-256          3) GOST-512          4) all
Choice (or 'm' to switch text/file mode, 't' to run test vectors, 'q' to quit): t

SHA-1  ""          passed
SHA-1  "abc"        passed
SHA-1  "abcbcbde...nopq" (56 bytes)    passed
SHA-1  1 000 000 x "a"                  passed
GOST-256 ""          passed
GOST-256 M1 (RFC 6986)                  passed
GOST-256 M2 (RFC 6986)                  passed
GOST-512 ""          passed
GOST-512 M1 (RFC 6986)                  passed
GOST-512 M2 (RFC 6986)                  passed
all 10 test vectors passed
```

Рисунок 2.1 – Меню программы и проверка на контрольных примерах

Хеш-функция отображает сообщение произвольной длины в хеш-код фиксированной длины. Криптографическая хеш-функция должна быть односторонней, то есть по хеш-коду нельзя найти прообраз, и стойкой к коллизиям, то есть нельзя найти два различных сообщения с одинаковым хеш-кодом. Оба изученных алгоритма построены по итеративной схеме: сообщение дополняется и разбивается на блоки по 512 бит, а блоки последовательно обрабатываются функцией сжатия, которая объединяет текущее состояние с очередным блоком.

2.1 Алгоритм SHA-1

Алгоритм SHA-1 работает с 32-битными словами, байты которых записываются от старшего к младшему, и вычисляет хеш-код длиной 160 бит. Сообщение длиной L бит дополняется единичным битом, нулями до длины, сравнимой с 448 по модулю 512, и 64-битной записью числа L . Дополнение добавляется всегда, поэтому сообщение, длина которого сравнима с 448–511 по модулю 512, получает лишний блок. Состояние из пяти слов $H_0, \dots, H_4$ инициализируется константами 67452301, EFCDAB89, 98BADCFE, 10325476, C3D2E1F0.

Каждый блок из 16 слов $W_0, \dots, W_{15}$ расширяется до 80 слов:

$$W_t = \text{ROTL}^1(W_{t-3} \oplus W_{t-8} \oplus W_{t-14} \oplus W_{t-16}), \quad t = 16, \dots, 79, \quad (2.1)$$

где ROTL^n – циклический сдвиг влево на n бит. Затем рабочие переменные a, b, c, d, e получают значения $H_0, \dots, H_4$ и проходят 80 раундов. В раунде t вычисляется

$$T = \text{ROTL}^5(a) + f_t(b, c, d) + e + K_t + W_t, \quad (2.2)$$

где сложение выполняется по модулю 2^{32} , после чего переменные сдвигаются:

$$(a, b, c, d, e) \leftarrow (T, a, \text{ROTL}^{30}(b), c, d). \quad (2.3)$$

Значение T вычисляется по значениям переменных до сдвига, поэтому в программе оно сохраняется во временной переменной до обновления остальных. Нелинейные функции и константы раундов приведены в таблице 2.1. После 80 раундов рабочие переменные прибавляются к $H_0, \dots, H_4$, а хеш-кодом является конкатенация $H_0 \parallel H_1 \parallel H_2 \parallel H_3 \parallel H_4$.

Таблица 2.1 – Функции и константы раундов SHA-1

Раунды	$f_t(b, c, d)$	K_t
0–19	$(b \wedge c) \vee (\neg b \wedge d)$	5A827999
20–39	$b \oplus c \oplus d$	6ED9EBA1
40–59	$(b \wedge c) \vee (b \wedge d) \vee (c \wedge d)$	8F1BBCDC
60–79	$b \oplus c \oplus d$	CA62C1D6

2.2 Алгоритм ГОСТ 34.11-2012

Алгоритм ГОСТ 34.11-2012 оперирует 512-битными векторами и вычисляет хеш-код длиной 512 или 256 бит. Функция сжатия построена из трёх преобразований вектора:

- S – замена каждого из 64 байтов по таблице π ;
- P – перестановка байтов τ , которая соответствует транспонированию матрицы байтов размера 8×8 ;
- L – умножение каждого из восьми 64-битных слов на матрицу A размера 64×64 над $\text{GF}(2)$: результат равен сумме по модулю 2 тех строк A , которым соответствуют единичные биты слова, причём старшему биту соответствует первая строка.

Композиция $\text{LPS} = L \circ P \circ S$ образует раунд, а $X(K, a) = K \oplus a$ – наложение раундового ключа. На их основе определяется 12-раундовое преобразование

$$E(K, m) = X(K_{13}, \text{LPS}(X(K_{12}, \dots \text{LPS}(X(K_1, m)) \dots))), \quad (2.4)$$

в котором раундовые ключи вычисляются последовательно: $K_1 = K$, $K_{i+1} = \text{LPS}(K_i \oplus C_i)$, где $C_1, \dots, C_{12}$ – 512-битные константы стандарта. Функция сжатия имеет вид

$$g_N(h, m) = E(\text{LPS}(h \oplus N), m) \oplus h \oplus m, \quad (2.5)$$

где N – число уже обработанных бит сообщения.

Хеширование начинается с вектора инициализации: нулевого для хеш-кода длиной 512 бит и состоящего из байтов 01 для 256 бит. Счётчик N и контрольная сумма Σ обнуляются. Пока длина сообщения M не меньше 512 бит, от него отделяются младшие 512 бит m и выполняются присваивания $h = g_N(h, m)$, $N = N + 512$ и $\Sigma = \Sigma + m$, где сложение выполняется по модулю 2^{512} . Оставшаяся часть сообщения, возможно пустая, дополняется до 512 бит вектором $0\dots0 \parallel 1$ и обрабатывается так же, но к N прибавляется её фактическая длина. В завершение вычисляются $h = g_0(h, N)$ и $h = g_0(h, \Sigma)$. Хеш-кодом длиной 512 бит является h , длиной 256 бит – старшие 256 бит h . Блок-схемы обоих алгоритмов приведены в приложении Б.

Стандарт записывает векторы от старшего байта к младшему, и модуль `hashing::gost34_11` хранит их в том же порядке, поэтому константы и таблицы перенесены из стандарта без изменений. Байтовая строка по соглашению читается в обратном порядке: её первый байт является младшим байтом вектора, и хеш-код выводится начиная с младшего байта. Так, в контрольном примере M1 из RFC 6986 строка «012...9012» записана как число 3231...3130, а хеш-код длиной 256 бит – как 00557BE5...1E159D. Поэтому при вычислении хеш-кода строки программа обращает порядок байтов сообщения и результата и выдаёт для M1 значение 9D151EEF...7B5500, то есть тот же хеш-код, прочитанный как байтовая строка. Это значение совпадает с результатом независимой реализации алгоритма.

Корректность реализации проверена на контрольных примерах, встроенных в программу: для SHA-1 это примеры из стандарта FIPS 180 и RFC 3174, включая сообщение из миллиона символов «а», для ГОСТ 34.11-2012 – примеры M1 и M2 из RFC 6986 и пустое сообщение. Дополнительно хеш-коды ГОСТ 34.11-2012 для сообщений длиной от 0 до 1000 байт, в том числе граничных длин 55, 56, 63, 64, 65, 127, 128 и 129 байт, сверены с независимой реализацией. Для файлов хеш-коды SHA-1 сверены с утилитой `shasum`, а хеш-коды ГОСТ 34.11-2012 – с той же независимой реализацией. Во всех случаях результаты совпадают.

Исходный код разработанного программного средства приведён в приложении А: реализация SHA-1 представлена в листинге А.1, реализация ГОСТ 34.11-2012 – в листинге А.2, точка входа в программу – в листинге А.3.

2.3 Хеширование текста

Результат работы программы для текста, вводимого с клавиатуры, представлен на рисунке 2.2. Сообщение «Привет, Стрибог!» в кодировке UTF-8

занимает 29 байт; хеш-коды всех трёх алгоритмов вычисляются для него за один раз. Для пустого сообщения программа выдаёт значения, приведённые в стандартах.

```
Choice (or 'm' to switch text/file mode, 't' to run test vectors, 'q' to quit): 4
Message: Привет, Стрибор!
message : 29 bytes
SHA-1    : AC3E2647BE25BED3B0CF5D63B37C7E8BF2D9D716 (160 bits)
GOST-256 : 43D717953B475B9E3D8092424D095A726DE8403659865AB2652642CA83FA4C0D (256 bits)
GOST-512 : E777E01D1C2329B40D2A82763CC236F1DB46DC1E6CFB58C0B6506C83D1EB0CE42788FCDEE31F3AC70DC2BF1
49DEFFCC4E9A1BC63716CDC58B8FFB7A20821EF66C (512 bits)

Hash functions - input/output: text
1) SHA-1          2) GOST-256          3) GOST-512          4) all
Choice (or 'm' to switch text/file mode, 't' to run test vectors, 'q' to quit): 4

Message:
message : 0 bytes
SHA-1    : DA39A3EE5E6B4B0D3255BFEF95601890AFD80709 (160 bits)
GOST-256 : 3F539A213E97C802CC229D474C6AA32A825A360B2A933A949FD925208D9CE1BB (256 bits)
GOST-512 : 8E945DA209AA869F0455928529BCAE4679E9873AB707B55315F56CEB98BEF0A7362F715528356EE83CDA5F2
AAC4C6AD2BA3A715C1BCD81CB8E9F90BF4C1C1A8A (512 bits)
```

Рисунок 2.2 – Хеширование текста

2.4 Лавинный эффект

Изменение одного символа сообщения меняет хеш-код целиком, как показано на рисунке 2.3. Замена «!» на «?» затрагивает 4 бита сообщения, а в хеш-коде SHA-1 при этом меняются 83 бита из 160, в хеш-коде ГОСТ 34.11-2012 длиной 256 бит – 114 бит из 256, то есть около половины, как и должно быть у хорошей хеш-функции.

```
Choice (or 'm' to switch text/file mode, 't' to run test vectors, 'q' to quit): 1
Message: Привет, Стрибор!
message : 29 bytes
SHA-1    : AC3E2647BE25BED3B0CF5D63B37C7E8BF2D9D716 (160 bits)

Hash functions - input/output: text
1) SHA-1          2) GOST-256          3) GOST-512          4) all
Choice (or 'm' to switch text/file mode, 't' to run test vectors, 'q' to quit): 1

Message: Привет, Стрибор?
message : 29 bytes
SHA-1    : B4468213C9F44DA0D1BF6DFD0D89ADBCC3A0B5CE (160 bits)

Hash functions - input/output: text
1) SHA-1          2) GOST-256          3) GOST-512          4) all
Choice (or 'm' to switch text/file mode, 't' to run test vectors, 'q' to quit): 2

Message: Привет, Стрибор!
message : 29 bytes
GOST-256 : 43D717953B475B9E3D8092424D095A726DE8403659865AB2652642CA83FA4C0D (256 bits)

Hash functions - input/output: text
1) SHA-1          2) GOST-256          3) GOST-512          4) all
Choice (or 'm' to switch text/file mode, 't' to run test vectors, 'q' to quit): 2

Message: Привет, Стрибор?
message : 29 bytes
GOST-256 : 11036EB13105D5D88E1ABB2FC33AD3D24E16489E49C8DFDB3A18813ECFEE59EB (256 bits)
```

Рисунок 2.3 – Хеш-коды сообщений, различающихся одним символом

2.5 Работа с файлами

Результат хеширования файлов представлен на рисунке 2.4. Файл отчёта к лабораторной работе №3 длиной 742 558 байт хешируется всеми алгоритмами, а затем его копия, в которой изменён один бит в байте 371 279. У копии меняются 78 бит из 160 хеш-кода SHA-1, 124 бита из 256 и 237 бит из 512 хеш-кодов ГОСТ 34.11-2012, поэтому хеш-код позволяет обнаружить любое изменение файла. Хеш-коды SHA-1 обоих файлов совпадают с результатом утилиты shasum.

```
Choice (or 'm' to switch text/file mode, 't' to run test vectors, 'q' to quit): m

Switched to file mode

Hash functions - input/output: file
1) SHA-1          2) GOST-256          3) GOST-512          4) all
Choice (or 'm' to switch text/file mode, 't' to run test vectors, 'q' to quit): 4

File: lab3.pdf
message : lab3.pdf (742558 bytes)
SHA-1   : C90BA060C35F5FA8C45B8B360FDEE0BBF20F6DF2 (160 bits)
GOST-256 : 7A20A55561702F81E32E84FB568D3A3B127A59AC99ACD6DC1945CD49B81CFC8 (256 bits)
GOST-512 : 6E38AFECECF02BAA0DFA58A72799CF7455EE3268BDC39943A1291E39B7D61B90C9A0AB65A2A2D4AAEE9FD01
7F6155AB317B33975531021493C831E885FD5C57C (512 bits)

Hash functions - input/output: file
1) SHA-1          2) GOST-256          3) GOST-512          4) all
Choice (or 'm' to switch text/file mode, 't' to run test vectors, 'q' to quit): 4

File: lab3_modified.pdf
message : lab3_modified.pdf (742558 bytes)
SHA-1   : E7D8FC91AC1A6AD9A6F8E187298AEFE7F3DAE18E (160 bits)
GOST-256 : F8A4282C52FFEDDF536A3CB7B909B4F43F57A81F19F83A8DE6D8986195017198 (256 bits)
GOST-512 : 1CF25A76F9C8C32194681F262F46AF22EDED7EAB4DA343E62EAB960C3F4D449CBD006BE2507F176F8EEE612C
5C9866CC89D1FEDF5DF16BA8DA1D67E69F22592A7 (512 bits)
```

Рисунок 2.4 – Хеширование файла и его изменённой копии

Таким образом хеш-функции SHA-1 и ГОСТ 34.11-2012 были изучены, реализованы и протестированы в полном объёме как для текста, вводимого с клавиатуры, так и для файлов.

ЗАКЛЮЧЕНИЕ

В ходе выполнения лабораторной работы были изучены принципы построения криптографических хеш-функций на примере алгоритмов SHA-1 и ГОСТ 34.11-2012. Было разработано программное средство на языке *Rust*, вычисляющее хеш-код SHA-1 длиной 160 бит и хеш-коды ГОСТ 34.11-2012 длиной 256 и 512 бит для текста, вводимого с клавиатуры, либо для файлов произвольного содержания.

Тестирование показало, что программа выдаёт значения из контрольных примеров стандартов, а изменение даже одного бита сообщения меняет около половины бит хеш-кода. Работа показала различие в построении алгоритмов. Функция сжатия SHA-1 состоит из 80 раундов простых операций над 32-битными словами: сложения, циклических сдвигов и поразрядных логических функций. Функция сжатия ГОСТ 34.11-2012 построена как блочный шифр *E* с 512-битным ключом, состоящий из нелинейной замены, перестановки и линейного преобразования, а кроме обработки блоков алгоритм учитывает общую длину сообщения N и контрольную сумму блоков Σ . Хеш-код SHA-1 длиной 160 бит сегодня считается недостаточно стойким: в 2017 году для него была построена первая коллизия, поэтому для новых систем применяются хеш-функции с большей длиной хеш-кода, такие как ГОСТ 34.11-2012.

Отдельно следует отметить отличия от изученных ранее алгоритмов. В отличие от шифров, хеш-функция не использует ключ и необратима: по хеш-коду нельзя восстановить сообщение. В отличие от имитовставки ГОСТ 28147-89, для вычисления хеш-кода не нужен секретный ключ, поэтому хеш-код сам по себе защищает только от случайных изменений данных, а для защиты от преднамеренных его необходимо передавать по защищённому каналу или подписывать.

Все задачи лабораторной работы выполнены в полном объёме.

ПРИЛОЖЕНИЕ А

(обязательное)

Листинг программного кода

Листинг А.1 – Реализация SHA-1 на *Rust*

```
const BLOCK_SIZE: usize = 64;
const ITERATIONS: usize = 80;

#[derive(Clone)]
struct ShalBuffer {
    a: u32,
    b: u32,
    c: u32,
    d: u32,
    e: u32,
}

pub struct Shal {}

impl Shal {
    pub fn hash(plain_bytes: &[u8]) -> Vec<u8> {
        let mut bytes = Vec::from(plain_bytes);

        let original_len = bytes.len() as u64 * 8;
        bytes.push(0x80);

        let padding = (56 + 64 - bytes.len() % 64) % 64;
        bytes.resize(bytes.len() + padding, 0u8);
        bytes.extend_from_slice(&original_len.to_be_bytes());

        let mut buffer = ShalBuffer {
            a: 0x67452301,
            b: 0xEFCDAB89,
            c: 0x98BADCFE,
            d: 0x10325476,
            e: 0xC3D2E1F0,
        };

        for block in bytes.chunks(BLOCK_SIZE) {
            let old_buffer = buffer.clone();

            let mut w: Vec<u32> = block
                .chunks(4)
                .map(|chunk| {
                    u32::from_be_bytes(chunk.try_into().expect("u32 should be
built from [u8; 4]"))
                })
                .collect();

            w.resize(ITERATIONS, 0);

            for i in 16..ITERATIONS {
                w[i] = (w[i - 16] ^ w[i - 14] ^ w[i - 8] ^ w[i -
3]).rotate_left(1);
            }

            let mut k: u32;
            let mut f_t: fn(u32, u32, u32) -> u32;
            for i in 0..ITERATIONS {
                match i {
                    0..20 => {
```

```

        f_t = |b, c, d| (b & c) | (!b & d);
        k = 0x5A827999;
    }
    20..40 => {
        f_t = |b, c, d| b ^ c ^ d;
        k = 0x6ED9EBA1;
    }
    40..60 => {
        f_t = |b, c, d| (b & c) | (b & d) | (c & d);
        k = 0x8F1BBCDC;
    }
    60..80 => {
        f_t = |b, c, d| b ^ c ^ d;
        k = 0xCA62C1D6;
    }
    _ => unreachable!(),
}

let t = buffer
    .a
    .rotate_left(5)
    .wrapping_add(f_t(buffer.b, buffer.c, buffer.d))
    .wrapping_add(buffer.e)
    .wrapping_add(k)
    .wrapping_add(w[i]);

buffer.e = buffer.d;
buffer.d = buffer.c;
buffer.c = buffer.b.rotate_left(30);
buffer.b = buffer.a;
buffer.a = t;
}

buffer.a = buffer.a.wrapping_add(old_buffer.a);
buffer.b = buffer.b.wrapping_add(old_buffer.b);
buffer.c = buffer.c.wrapping_add(old_buffer.c);
buffer.d = buffer.d.wrapping_add(old_buffer.d);
buffer.e = buffer.e.wrapping_add(old_buffer.e);
}

let mut result: Vec<u8> = vec![];
result.extend_from_slice(&buffer.a.to_be_bytes());
result.extend_from_slice(&buffer.b.to_be_bytes());
result.extend_from_slice(&buffer.c.to_be_bytes());
result.extend_from_slice(&buffer.d.to_be_bytes());
result.extend_from_slice(&buffer.e.to_be_bytes());

return result;
}
}

```

Листинг А.2 – Реализация ГОСТ 34.11-2012 на *Rust*

```

const BLOCK_SIZE: usize = 64;

const PI: [u8; 256] = [
    252, 238, 221, 17, 207, 110, 49, 22, 251, 196, 250, 218, 35, 197, 4, 77,
    233, 119, 240, 219,
    147, 46, 153, 186, 23, 54, 241, 187, 20, 205, 95, 193, 249, 24, 101, 90,
    226, 92, 239, 33, 129,
    28, 60, 66, 139, 1, 142, 79, 5, 132, 2, 174, 227, 106, 143, 160, 6, 11,
    237, 152, 127, 212,
    211, 31, 235, 52, 44, 81, 234, 200, 72, 171, 242, 42, 104, 162, 253, 58,

```

```

206, 204, 181, 112,
    14, 86, 8, 12, 118, 18, 191, 114, 19, 71, 156, 183, 93, 135, 21, 161,
150, 41, 16, 123, 154,
    199, 243, 145, 120, 111, 157, 158, 178, 177, 50, 117, 25, 61, 255, 53,
138, 126, 109, 84, 198,
    128, 195, 189, 13, 87, 223, 245, 36, 169, 62, 168, 67, 201, 215, 121,
214, 246, 124, 34, 185,
    3, 224, 15, 236, 222, 122, 148, 176, 188, 220, 232, 40, 80, 78, 51, 10,
74, 167, 151, 96, 115,
    30, 0, 98, 68, 26, 184, 56, 130, 100, 159, 38, 65, 173, 69, 70, 146, 39,
94, 85, 47, 140, 163,
    165, 125, 105, 213, 149, 59, 7, 88, 179, 64, 134, 172, 29, 247, 48, 55,
107, 228, 136, 217,
    231, 137, 225, 27, 131, 73, 76, 63, 248, 254, 141, 83, 170, 144, 202,
216, 133, 97, 32, 113,
    103, 164, 45, 43, 9, 91, 203, 155, 37, 208, 190, 229, 108, 82, 89, 166,
116, 210, 230, 244,
    180, 192, 209, 102, 175, 194, 57, 75, 99, 182,
];

```

```

const A: [u64; BLOCK_SIZE] = [
    0x8e20faa72ba0b470,
    0x47107ddd9b505a38,
    0xad08b0e0c3282d1c,
    0xd8045870ef14980e,
    0x6c022c38f90a4c07,
    0x3601161cf205268d,
    0x1b8e0b0e798c13c8,
    0x83478b07b2468764,
    0xa011d380818e8f40,
    0x5086e740ce47c920,
    0x2843fd2067adea10,
    0x14aff010bdd87508,
    0x0ad97808d06cb404,
    0x05e23c0468365a02,
    0x8c711e02341b2d01,
    0x46b60f011a83988e,
    0x90dab52a387ae76f,
    0x486dd4151c3dfdb9,
    0x24b86a840e90f0d2,
    0x125c354207487869,
    0x092e94218d243cba,
    0x8a174a9ec8121e5d,
    0x4585254f64090fa0,
    0xaccc9ca9328a8950,
    0x9d4df05d5f661451,
    0xc0a878a0a1330aa6,
    0x60543c50de970553,
    0x302a1e286fc58ca7,
    0x18150f14b9ec46dd,
    0x0c84890ad27623e0,
    0x0642ca05693b9f70,
    0x0321658cba93c138,
    0x86275df09ce8aaa8,
    0x439da0784e745554,
    0xafc0503c273aa42a,
    0xd960281e9d1d5215,
    0xe230140fc0802984,
    0x71180a8960409a42,
    0xb60c05ca30204d21,
    0x5b068c651810a89e,
    0x456c34887a3805b9,
    0xac361a443d1c8cd2,

```

```

0x561b0d22900e4669,
0x2b838811480723ba,
0x9bcf4486248d9f5d,
0xc3e9224312c8c1a0,
0xeffa11af0964ee50,
0xf97d86d98a327728,
0xe4fa2054a80b329c,
0x727d102a548b194e,
0x39b008152acb8227,
0x9258048415eb419d,
0x492c024284fbaec0,
0xaa16012142f35760,
0x550b8e9e21f7a530,
0xa48b474f9ef5dc18,
0x70a6a56e2440598e,
0x3853dc371220a247,
0x1ca76e95091051ad,
0x0edd37c48a08a6d8,
0x07e095624504536c,
0x8d70c431ac02a736,
0xc83862965601dd1b,
0x641c314b2b8ee083,
];

const TAU: [usize; BLOCK_SIZE] = [
    0, 8, 16, 24, 32, 40, 48, 56, 1, 9, 17, 25, 33, 41, 49, 57, 2, 10, 18,
    26, 34, 42, 50, 58, 3,
    11, 19, 27, 35, 43, 51, 59, 4, 12, 20, 28, 36, 44, 52, 60, 5, 13, 21, 29,
    37, 45, 53, 61, 6,
    14, 22, 30, 38, 46, 54, 62, 7, 15, 23, 31, 39, 47, 55, 63,
];

pub const C: [[u8; 64]; 12] = [
    [
        0xb1, 0x08, 0x5b, 0xda, 0x1e, 0xca, 0xda, 0xe9, 0xeb, 0xcb, 0x2f,
        0x81, 0xc0, 0x65, 0x7c,
        0x1f, 0x2f, 0x6a, 0x76, 0x43, 0x2e, 0x45, 0xd0, 0x16, 0x71, 0x4e,
        0xb8, 0x8d, 0x75, 0x85,
        0xc4, 0xfc, 0x4b, 0x7c, 0xe0, 0x91, 0x92, 0x67, 0x69, 0x01, 0xa2,
        0x42, 0x2a, 0x08, 0xa4,
        0x60, 0xd3, 0x15, 0x05, 0x76, 0x74, 0x36, 0xcc, 0x74, 0x4d, 0x23,
        0xdd, 0x80, 0x65, 0x59,
        0xf2, 0xa6, 0x45, 0x07,
    ],
    [
        0x6f, 0xa3, 0xb5, 0x8a, 0xa9, 0x9d, 0x2f, 0x1a, 0x4f, 0xe3, 0x9d,
        0x46, 0x0f, 0x70, 0xb5,
        0xd7, 0xf3, 0xfe, 0xea, 0x72, 0x0a, 0x23, 0x2b, 0x98, 0x61, 0xd5,
        0x5e, 0x0f, 0x16, 0xb5,
        0x01, 0x31, 0x9a, 0xb5, 0x17, 0x6b, 0x12, 0xd6, 0x99, 0x58, 0x5c,
        0xb5, 0x61, 0xc2, 0xdb,
        0x0a, 0xa7, 0xca, 0x55, 0xdd, 0xa2, 0x1b, 0xd7, 0xcb, 0xcd, 0x56,
        0xe6, 0x79, 0x04, 0x70,
        0x21, 0xb1, 0x9b, 0xb7,
    ],
    [
        0xf5, 0x74, 0xdc, 0xac, 0x2b, 0xce, 0x2f, 0xc7, 0x0a, 0x39, 0xfc,
        0x28, 0x6a, 0x3d, 0x84,
        0x35, 0x06, 0xf1, 0x5e, 0x5f, 0x52, 0x9c, 0x1f, 0x8b, 0xf2, 0xea,
        0x75, 0x14, 0xb1, 0x29,
        0x7b, 0x7b, 0xd3, 0xe2, 0x0f, 0xe4, 0x90, 0x35, 0x9e, 0xb1, 0xc1,
        0xc9, 0x3a, 0x37, 0x60,
        0x62, 0xdb, 0x09, 0xc2, 0xb6, 0xf4, 0x43, 0x86, 0x7a, 0xdb, 0x31,
    ],
];

```

```

0x99, 0x1e, 0x96, 0xf5,
    0x0a, 0xba, 0x0a, 0xb2,
],
[
    0xef, 0x1f, 0xdf, 0xb3, 0xe8, 0x15, 0x66, 0xd2, 0xf9, 0x48, 0xe1,
0xa0, 0x5d, 0x71, 0xe4,
    0xdd, 0x48, 0x8e, 0x85, 0x7e, 0x33, 0x5c, 0x3c, 0x7d, 0x9d, 0x72,
0x1c, 0xad, 0x68, 0x5e,
    0x35, 0x3f, 0xa9, 0xd7, 0x2c, 0x82, 0xed, 0x03, 0xd6, 0x75, 0xd8,
0xb7, 0x13, 0x33, 0x93,
    0x52, 0x03, 0xbe, 0x34, 0x53, 0xea, 0xa1, 0x93, 0xe8, 0x37, 0xf1,
0x22, 0x0c, 0xbe, 0xbc,
    0x84, 0xe3, 0xd1, 0x2e,
],
[
    0x4b, 0xea, 0x6b, 0xac, 0xad, 0x47, 0x47, 0x99, 0x9a, 0x3f, 0x41,
0x0c, 0x6c, 0xa9, 0x23,
    0x63, 0x7f, 0x15, 0x1c, 0x1f, 0x16, 0x86, 0x10, 0x4a, 0x35, 0x9e,
0x35, 0xd7, 0x80, 0x0f,
    0xff, 0xbd, 0xbf, 0xcd, 0x17, 0x47, 0x25, 0x3a, 0xf5, 0xa3, 0xdf,
0xff, 0x00, 0xb7, 0x23,
    0x27, 0x1a, 0x16, 0x7a, 0x56, 0xa2, 0x7e, 0xa9, 0xea, 0x63, 0xf5,
0x60, 0x17, 0x58, 0xfd,
    0x7c, 0x6c, 0xfe, 0x57,
],
[
    0xae, 0x4f, 0xae, 0xae, 0x1d, 0x3a, 0xd3, 0xd9, 0x6f, 0xa4, 0xc3,
0x3b, 0x7a, 0x30, 0x39,
    0xc0, 0x2d, 0x66, 0xc4, 0xf9, 0x51, 0x42, 0xa4, 0x6c, 0x18, 0x7f,
0x9a, 0xb4, 0x9a, 0xf0,
    0x8e, 0xc6, 0xcf, 0xfa, 0xa6, 0xb7, 0x1c, 0x9a, 0xb7, 0xb4, 0x0a,
0xf2, 0x1f, 0x66, 0xc2,
    0xbe, 0xc6, 0xb6, 0xbf, 0x71, 0xc5, 0x72, 0x36, 0x90, 0x4f, 0x35,
0xfa, 0x68, 0x40, 0x7a,
    0x46, 0x64, 0x7d, 0x6e,
],
[
    0xf4, 0xc7, 0x0e, 0x16, 0xee, 0xaa, 0xc5, 0xec, 0x51, 0xac, 0x86,
0xfe, 0xbf, 0x24, 0x09,
    0x54, 0x39, 0x9e, 0xc6, 0xc7, 0xe6, 0xbf, 0x87, 0xc9, 0xd3, 0x47,
0x3e, 0x33, 0x19, 0x7a,
    0x93, 0xc9, 0x09, 0x92, 0xab, 0xc5, 0x2d, 0x82, 0x2c, 0x37, 0x06,
0x47, 0x69, 0x83, 0x28,
    0x4a, 0x05, 0x04, 0x35, 0x17, 0x45, 0x4c, 0xa2, 0x3c, 0x4a, 0xf3,
0x88, 0x86, 0x56, 0x4d,
    0x3a, 0x14, 0xd4, 0x93,
],
[
    0x9b, 0x1f, 0x5b, 0x42, 0x4d, 0x93, 0xc9, 0xa7, 0x03, 0xe7, 0xaa,
0x02, 0x0c, 0x6e, 0x41,
    0x41, 0x4e, 0xb7, 0xf8, 0x71, 0x9c, 0x36, 0xde, 0x1e, 0x89, 0xb4,
0x44, 0x3b, 0x4d, 0xdb,
    0xc4, 0x9a, 0xf4, 0x89, 0x2b, 0xcb, 0x92, 0x9b, 0x06, 0x90, 0x69,
0xd1, 0x8d, 0x2b, 0xd1,
    0xa5, 0xc4, 0x2f, 0x36, 0xac, 0xc2, 0x35, 0x59, 0x51, 0xa8, 0xd9,
0xa4, 0x7f, 0x0d, 0xd4,
    0xbf, 0x02, 0xe7, 0x1e,
],
[
    0x37, 0x8f, 0x5a, 0x54, 0x16, 0x31, 0x22, 0x9b, 0x94, 0x4c, 0x9a,
0xd8, 0xec, 0x16, 0x5f,
    0xde, 0x3a, 0x7d, 0x3a, 0x1b, 0x25, 0x89, 0x42, 0x24, 0x3c, 0xd9,
0x55, 0xb7, 0xe0, 0x0d,

```

```

        0x09, 0x84, 0x80, 0x0a, 0x44, 0x0b, 0xdb, 0xb2, 0xce, 0xb1, 0x7b,
0x2b, 0x8a, 0x9a, 0xa6,
        0x07, 0x9c, 0x54, 0x0e, 0x38, 0xdc, 0x92, 0xcb, 0x1f, 0x2a, 0x60,
0x72, 0x61, 0x44, 0x51,
        0x83, 0x23, 0x5a, 0xdb,
    ],
    [
        0xab, 0xbe, 0xde, 0xa6, 0x80, 0x05, 0x6f, 0x52, 0x38, 0x2a, 0xe5,
0x48, 0xb2, 0xe4, 0xf3,
        0xf3, 0x89, 0x41, 0xe7, 0x1c, 0xff, 0x8a, 0x78, 0xdb, 0x1f, 0xff,
0xe1, 0x8a, 0x1b, 0x33,
        0x61, 0x03, 0x9f, 0xe7, 0x67, 0x02, 0xaf, 0x69, 0x33, 0x4b, 0x7a,
0x1e, 0x6c, 0x30, 0x3b,
        0x76, 0x52, 0xf4, 0x36, 0x98, 0xfa, 0xd1, 0x15, 0x3b, 0xb6, 0xc3,
0x74, 0xb4, 0xc7, 0xfb,
        0x98, 0x45, 0x9c, 0xed,
    ],
    [
        0x7b, 0xcd, 0x9e, 0xd0, 0xef, 0xc8, 0x89, 0xfb, 0x30, 0x02, 0xc6,
0xcd, 0x63, 0x5a, 0xfe,
        0x94, 0xd8, 0xfa, 0x6b, 0xbb, 0xeb, 0xab, 0x07, 0x61, 0x20, 0x01,
0x80, 0x21, 0x14, 0x84,
        0x66, 0x79, 0x8a, 0x1d, 0x71, 0xef, 0xea, 0x48, 0xb9, 0xca, 0xef,
0xba, 0xcd, 0x1d, 0x7d,
        0x47, 0x6e, 0x98, 0xde, 0xa2, 0x59, 0x4a, 0xc0, 0x6f, 0xd8, 0x5d,
0x6b, 0xca, 0xa4, 0xcd,
        0x81, 0xf3, 0x2d, 0x1b,
    ],
    [
        0x37, 0x8e, 0xe7, 0x67, 0xf1, 0x16, 0x31, 0xba, 0xd2, 0x13, 0x80,
0xb0, 0x04, 0x49, 0xb1,
        0x7a, 0xcd, 0xa4, 0x3c, 0x32, 0xbc, 0xdf, 0x1d, 0x77, 0xf8, 0x20,
0x12, 0xd4, 0x30, 0x21,
        0x9f, 0x9b, 0x5d, 0x80, 0xef, 0x9d, 0x18, 0x91, 0xcc, 0x86, 0xe7,
0x1d, 0xaa, 0xaa, 0x88,
        0xe1, 0x28, 0x52, 0xfa, 0xf4, 0x17, 0xd5, 0xd9, 0xb2, 0x1b, 0x99,
0x48, 0xbc, 0x92, 0x4a,
        0xf1, 0x1b, 0xd7, 0x20,
    ],
],
];

pub struct Gost34_11 {}

impl Gost34_11 {
    fn xor(a: &[u8], b: &[u8]) -> Vec<u8> {
        debug_assert!(a.len() == b.len());
        return a.iter().zip(b.iter()).map(|(&x, &y)| x ^ y).collect();
    }

    fn p(a: &[u8]) -> Vec<u8> {
        let mut output = Vec::from([0u8; BLOCK_SIZE]);
        for i in 0..BLOCK_SIZE {
            output[i] = a[TAU[i]];
        }

        return output;
    }

    fn s(a: &[u8]) -> Vec<u8> {
        debug_assert!(a.len() == BLOCK_SIZE);
        return a.iter().map(|&x| PI[x as usize]).collect();
    }
}

```

```

fn l(a: &[u8]) -> Vec<u8> {
    debug_assert!(a.len() == BLOCK_SIZE);
    let mut res: Vec<u8> = vec![];
    for block in a.chunks(8) {
        let mut res_i = 0u64;
        let n = u64::from_be_bytes(
            block
                .try_into()
                .expect("u64 can be constructed only with [u8; 8]"),
        );
        for i in 0..64 {
            if (n >> i) & 1 == 1 {
                res_i ^= A[63 - i];
            }
        }

        res.extend_from_slice(&res_i.to_be_bytes());
    }

    return res;
}

fn key_schedule(k: &[u8], i: usize) -> Vec<u8> {
    let state = Self::xor(k, &C[i]);
    return Self::l(&Self::p(&Self::s(&state)));
}

fn e(original_k: &[u8], m: &[u8]) -> Vec<u8> {
    let mut state = Self::xor(original_k, m);
    let mut k: Vec<u8> = Vec::from(original_k);
    for i in 0..=11 {
        state = Self::l(&Self::p(&Self::s(&state)));
        k = Self::key_schedule(&k, i);
        state = Self::xor(&state, &k);
    }

    return state;
}

fn g(n: &[u8], m: &[u8], h: &[u8]) -> Vec<u8> {
    let mut k: Vec<u8> = Self::xor(&h, &n);
    k = Self::l(&Self::p(&Self::s(&k)));
    let mut t = Self::e(&k, m);
    t = Self::xor(&t, h);
    return Self::xor(&t, m);
}

fn add_mod_512(a: &[u8], b: &[u8]) -> Vec<u8> {
    debug_assert_eq!(a.len(), BLOCK_SIZE);
    debug_assert_eq!(b.len(), BLOCK_SIZE);

    let mut result = vec![0u8; BLOCK_SIZE];
    let mut carry = 0u16;

    for i in (0..BLOCK_SIZE).rev() {
        let sum = a[i] as u16 + b[i] as u16 + carry;
        result[i] = sum as u8;
        carry = sum >> 8;
    }

    return result;
}

```

```

fn from_u64(value: u64) -> Vec<u8> {
    let mut result = vec![0u8; BLOCK_SIZE];
    result[(BLOCK_SIZE - 8)..].copy_from_slice(&value.to_be_bytes());
    return result;
}

fn final_hash(mut h: Vec<u8>, n: Vec<u8>, sigma: Vec<u8>) -> Vec<u8> {
    let zero = [0u8; BLOCK_SIZE];

    h = Self::g(&zero, &n, &h);
    h = Self::g(&zero, &sigma, &h);

    return h;
}

pub fn hash_512(bytes: &[u8]) -> Vec<u8> {
    let mut h = vec![0u8; BLOCK_SIZE];
    let mut n = vec![0u8; BLOCK_SIZE];
    let mut sigma = vec![0u8; BLOCK_SIZE];

    let mut end = bytes.len();

    while end >= BLOCK_SIZE {
        let start = end - BLOCK_SIZE;
        let m = &bytes[start..end];

        h = Self::g(&n, m, &h);
        n = Self::add_mod_512(&n, &Self::from_u64(512));
        sigma = Self::add_mod_512(&sigma, m);

        end = start;
    }

    let mut m = vec![0u8; BLOCK_SIZE];

    m[BLOCK_SIZE - end - 1] = 0x01;
    m[BLOCK_SIZE - end..].copy_from_slice(&bytes[..end]);

    h = Self::g(&n, &m, &h);
    n = Self::add_mod_512(&n, &Self::from_u64((end * 8) as u64));
    sigma = Self::add_mod_512(&sigma, &m);

    return Self::final_hash(h, n, sigma);
}

pub fn hash_256(bytes: &[u8]) -> Vec<u8> {
    let mut h = vec![0x01u8; BLOCK_SIZE];
    let mut n = vec![0u8; BLOCK_SIZE];
    let mut sigma = vec![0u8; BLOCK_SIZE];

    let mut end = bytes.len();

    while end >= BLOCK_SIZE {
        let start = end - BLOCK_SIZE;
        let m = &bytes[start..end];

        h = Self::g(&n, m, &h);
        n = Self::add_mod_512(&n, &Self::from_u64(512));
        sigma = Self::add_mod_512(&sigma, m);

        end = start;
    }
}

```

```

        let mut m = vec![0u8; BLOCK_SIZE];

        m[BLOCK_SIZE - end - 1] = 0x01;
        m[BLOCK_SIZE - end..].copy_from_slice(&bytes[..end]);

        h = Self::g(&n, &m, &h);
        n = Self::add_mod_512(&n, &Self::from_u64((end * 8) as u64));
        sigma = Self::add_mod_512(&sigma, &m);

        h = Self::final_hash(h, n, sigma);

        return h[..32].to_vec();
    }
}

```

Листинг А.3 – Точка входа в программу

```

mod hashing;
use common::cli::{self, IoMode};
use hashing::gost34_11::Gost34_11;
use hashing::sha1::Sha1;

const TITLE: &str = "Hash functions";

struct Algorithm {
    name: &'static str,
    hash: fn(&[u8]) -> Vec<u8>,
}

const ALGORITHMS: [Algorithm; 3] = [
    Algorithm {
        name: "SHA-1",
        hash: Sha1::hash,
    },
    Algorithm {
        name: "GOST-256",
        hash: gost_256,
    },
    Algorithm {
        name: "GOST-512",
        hash: gost_512,
    },
];

fn as_byte_string(hash: fn(&[u8]) -> Vec<u8>, bytes: &[u8]) -> Vec<u8> {
    let message: Vec<u8> = bytes.iter().rev().copied().collect();
    let mut digest = hash(&message);
    digest.reverse();

    return digest;
}

fn gost_256(bytes: &[u8]) -> Vec<u8> {
    return as_byte_string(Gost34_11::hash_256, bytes);
}

fn gost_512(bytes: &[u8]) -> Vec<u8> {
    return as_byte_string(Gost34_11::hash_512, bytes);
}

const M2: &str =
"d1e520e2e5f2f0e82c20d1f2f0e8e1eeee6e820e2edf3f6e82c20e2e5fef2fa20f120ecee\
f0ff20f1f2f0e5ebe0ece820ede020f5f0e0elf0fbff20efebfaeafb20c8e3eef0e5e2fb";

```

```

struct TestVector {
    algorithm: usize,
    label: &'static str,
    message: fn() -> Vec<u8>,
    digest: &'static str,
}

const TEST_VECTORS: [TestVector; 10] = [
    TestVector {
        algorithm: 0,
        label: "\"\\\"",
        message: || Vec::new(),
        digest: "DA39A3EE5E6B4B0D3255BFEF95601890AFD80709",
    },
    TestVector {
        algorithm: 0,
        label: "\"abc\"",
        message: || b"abc".to_vec(),
        digest: "A9993E364706816ABA3E25717850C26C9CD0D89D",
    },
    TestVector {
        algorithm: 0,
        label: "\"abdbcde...nopq\" (56 bytes)",
        message: ||
b"abdbcdecdefdefgefghfghighijhiijkijklklmklmnlmnomnopnopq".to_vec(),
        digest: "84983E441C3BD26EBAAE4AA1F95129E5E54670F1",
    },
    TestVector {
        algorithm: 0,
        label: "1 000 000 x \"a\"",
        message: || vec![b'a'; 1_000_000],
        digest: "34AA973CD4C4DAA4F61EEB2BDBAD27316534016F",
    },
    TestVector {
        algorithm: 1,
        label: "\"\\\"",
        message: || Vec::new(),
        digest:
"3F539A213E97C802CC229D474C6AA32A825A360B2A933A949FD925208D9CE1BB",
    },
    TestVector {
        algorithm: 1,
        label: "M1 (RFC 6986)",
        message: ||
b"012345678901234567890123456789012345678901234567890123456789012".to_vec(),
        digest:
"9D151EEFD8590B89DAA6BA6CB74AF9275DD051026BB149A452FD84E5E57B5500",
    },
    TestVector {
        algorithm: 1,
        label: "M2 (RFC 6986)",
        message: || cli::from_hex(M2).expect("M2 is valid hex"),
        digest:
"9DD2FE4E90409E5DA87F53976D7405B0C0CAC628FC669A741D50063C557E8F50",
    },
    TestVector {
        algorithm: 2,
        label: "\"\\\"",
        message: || Vec::new(),
        digest:
"8E945DA209AA869F0455928529BCAE4679E9873AB707B55315F56CEB98BEF0A7\

```

```

362F715528356EE83CDA5F2AAC4C6AD2BA3A715C1BCD81CB8E9F90BF4C1C1A8A",
    },
    TestVector {
        algorithm: 2,
        label: "M1 (RFC 6986)",
        message: ||
b"012345678901234567890123456789012345678901234567890123456789012".to_vec(),
        digest:
"1B54D01A4AF5B9D5CC3D86D68D285462B19ABC2475222F35C085122BE4BA1FFA\
00AD30F8767B3A82384C6574F024C311E2A481332B08EF7F41797891C1646F48",
    },
    TestVector {
        algorithm: 2,
        label: "M2 (RFC 6986)",
        message: || cli::from_hex(M2).expect("M2 is valid hex"),
        digest:
"1E88E62226BFCA6F9994F1F2D51569E0DAF8475A3B0FE61A5300EEE46D961376\
035FE83549ADA2B8620FCD7C496CE5B33F0CB9DDDC2B6460143B03DABAC9FB28",
    },
];

fn run_test_vectors() -> Result<(), String> {
    let mut failed = 0;
    for vector in &TEST_VECTORS {
        let algorithm = &ALGORITHMS[vector.algorithm];
        let digest = cli::to_hex(&(algorithm.hash) (&(vector.message) ()));
        let passed = digest == vector.digest;
        if !passed {
            failed += 1;
        }

        println!(
            "  {:<8} {:<30} {}",
            algorithm.name,
            vector.label,
            if passed { "passed" } else { "FAILED" }
        );
        if !passed {
            println!("    expected : {}", vector.digest);
            println!("    got      : {digest}");
        }
    }

    if failed > 0 {
        return Err(format!(
            "{failed} of {} test vectors failed",
            TEST_VECTORS.len()
        ));
    }

    println!("  all {} test vectors passed", TEST_VECTORS.len());

    return Ok(());
}

fn read_message(io_mode: IoMode) -> Result<(Vec<u8>, String), String> {
    match io_mode {
        IoMode::Text => {
            let bytes = cli::prompt("Message: ").into_bytes();
            let description = format!("{}", bytes.len());

```

```

        return Ok((bytes, description));
    }
    IoMode::File => {
        let path = cli::ask_input_path("File: ")?;
        let bytes = cli::read_file(&path)?;
        let description = format!("{}", ({} bytes)", path.display(),
bytes.len());

        return Ok((bytes, description));
    }
}

fn run_hash(io_mode: IoMode, algorithms: &[Algorithm]) -> Result<(), String>
{
    let (message, description) = read_message(io_mode)?;

    println!(" message : {description}");
    for algorithm in algorithms {
        let digest = (algorithm.hash)(&message);
        println!(
            "  {:<9}: {} ({} bits)",
            algorithm.name,
            cli::to_hex(&digest),
            digest.len() * 8
        );
    }

    return Ok(());
}

fn main() {
    let mut io_mode = IoMode::Text;

    loop {
        println!();
        println!("{}", TITLE} - input/output: {}", io_mode.name());
        println!(
            "{:<22}{:<22}{:<22}4) all",
            "1) SHA-1", "2) GOST-256", "3) GOST-512"
        );

        let choice = cli::prompt(
            "Choice (or 'm' to switch text/file mode, 't' to run test
vectors, 'q' to quit): ",
        );
        println!();

        let result = match choice.as_str() {
            "1" | "2" | "3" => {
                let idx = choice.parse:::<usize>().expect("matched a digit") -
1;

                run_hash(io_mode, &ALGORITHMS[idx..=idx])
            }
            "4" => run_hash(io_mode, &ALGORITHMS),
            "m" => {
                io_mode = io_mode.toggled();
                println!("Switched to {} mode", io_mode.name());
                Ok(())
            }
            "t" => run_test_vectors(),
            "q" => break,
            _ => Err(format!("unknown choice: {choice:?}")),
        };
    }
}

```

```
};  
  
if let Err(err) = result {  
    println!("Error: {err}");  
}  
}
```

ПРИЛОЖЕНИЕ Б **(обязательное)** **Блок-схемы алгоритмов**

Блок-схема алгоритма SHA-1

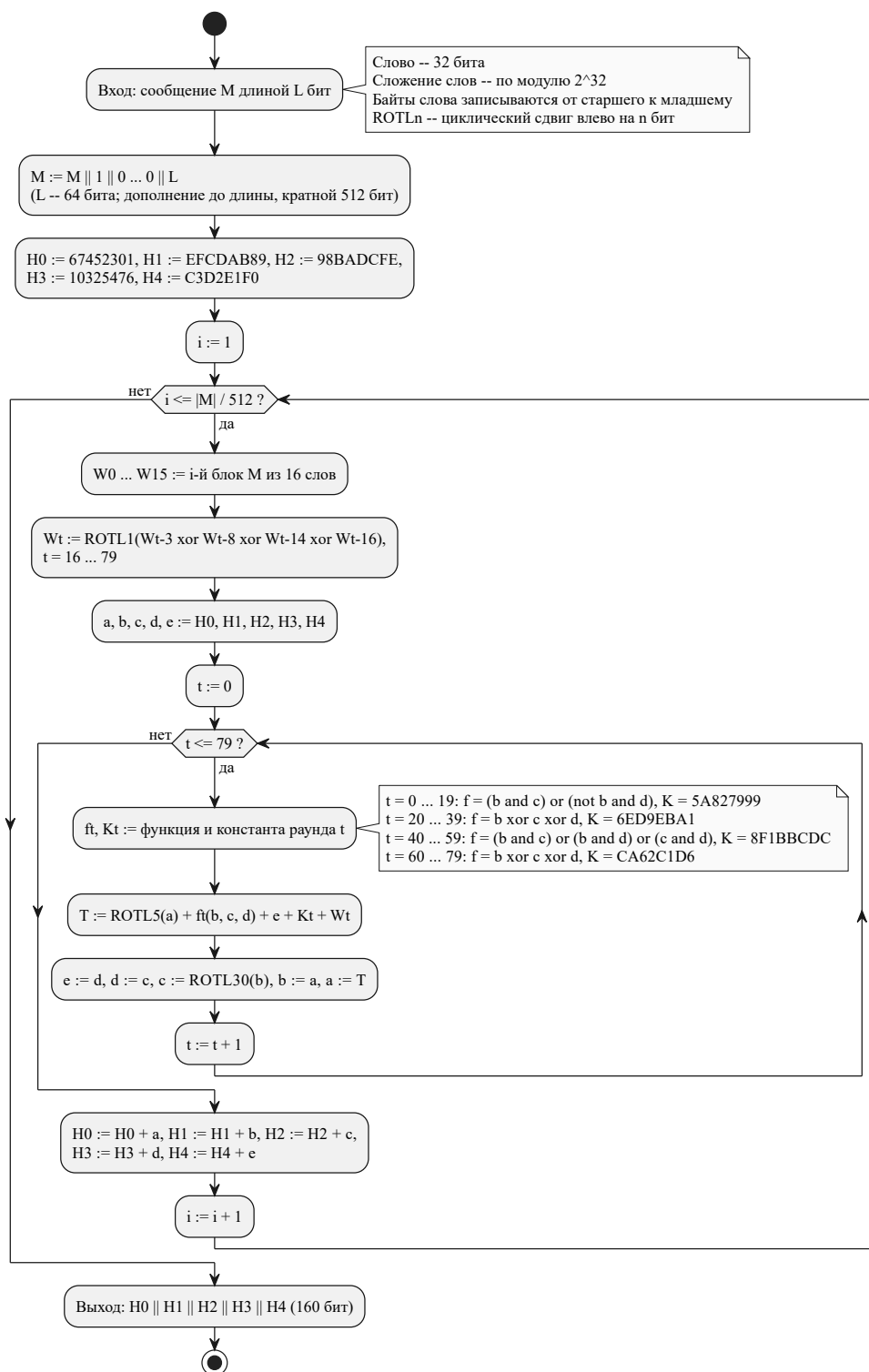


Рисунок Б.1 – Блок-схема алгоритма SHA-1

Блок-схема алгоритма ГОСТ 34.11-2012

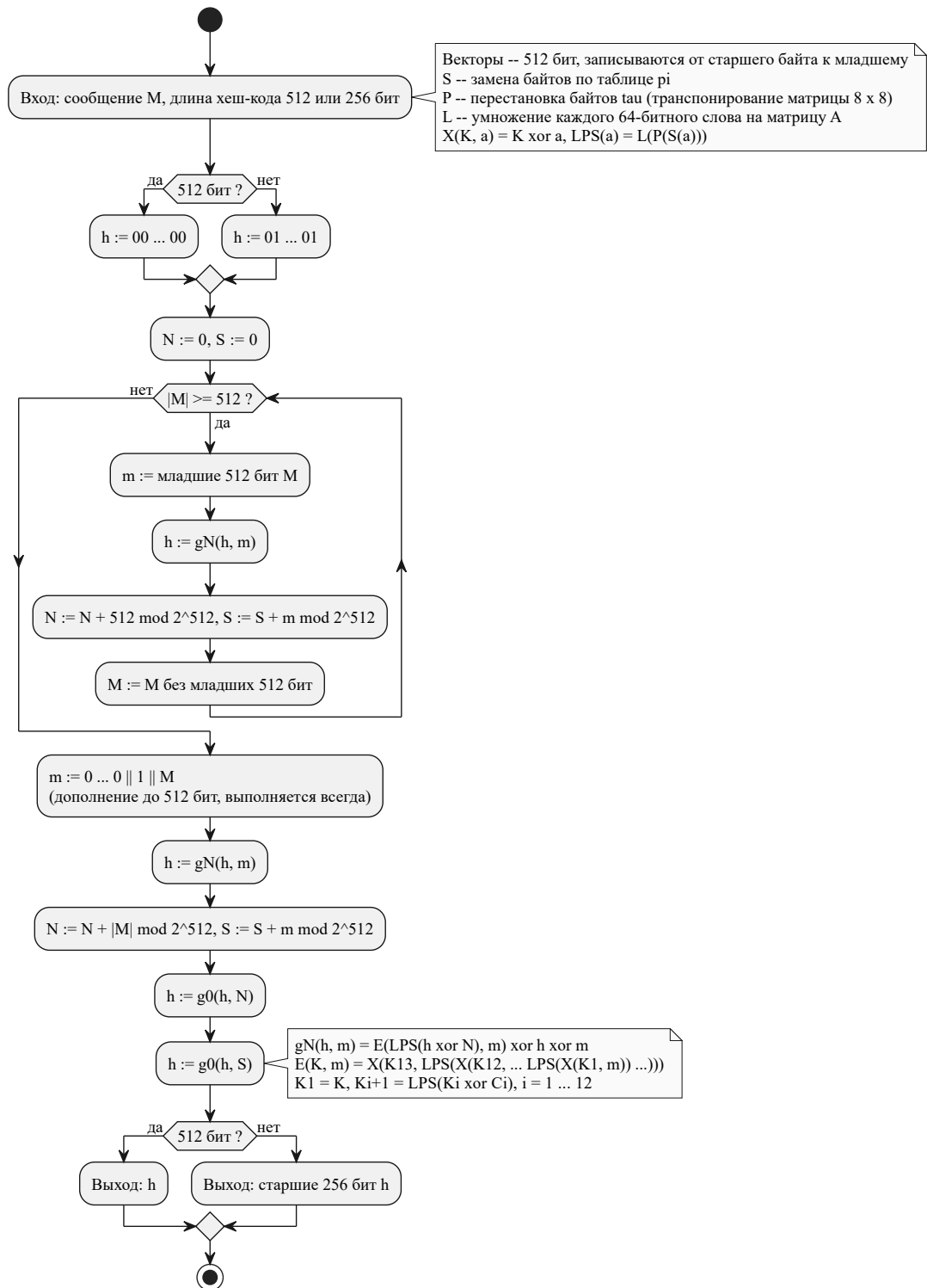


Рисунок Б.2 – Блок-схема алгоритма ГОСТ 34.11-2012